

TEAM 2

TLSE RACING HACKATHON

AUTONOMOUS RACING CHALLENGE



Nov 06th 2025

PRESENTATION OUTLINE

ESTIMATION

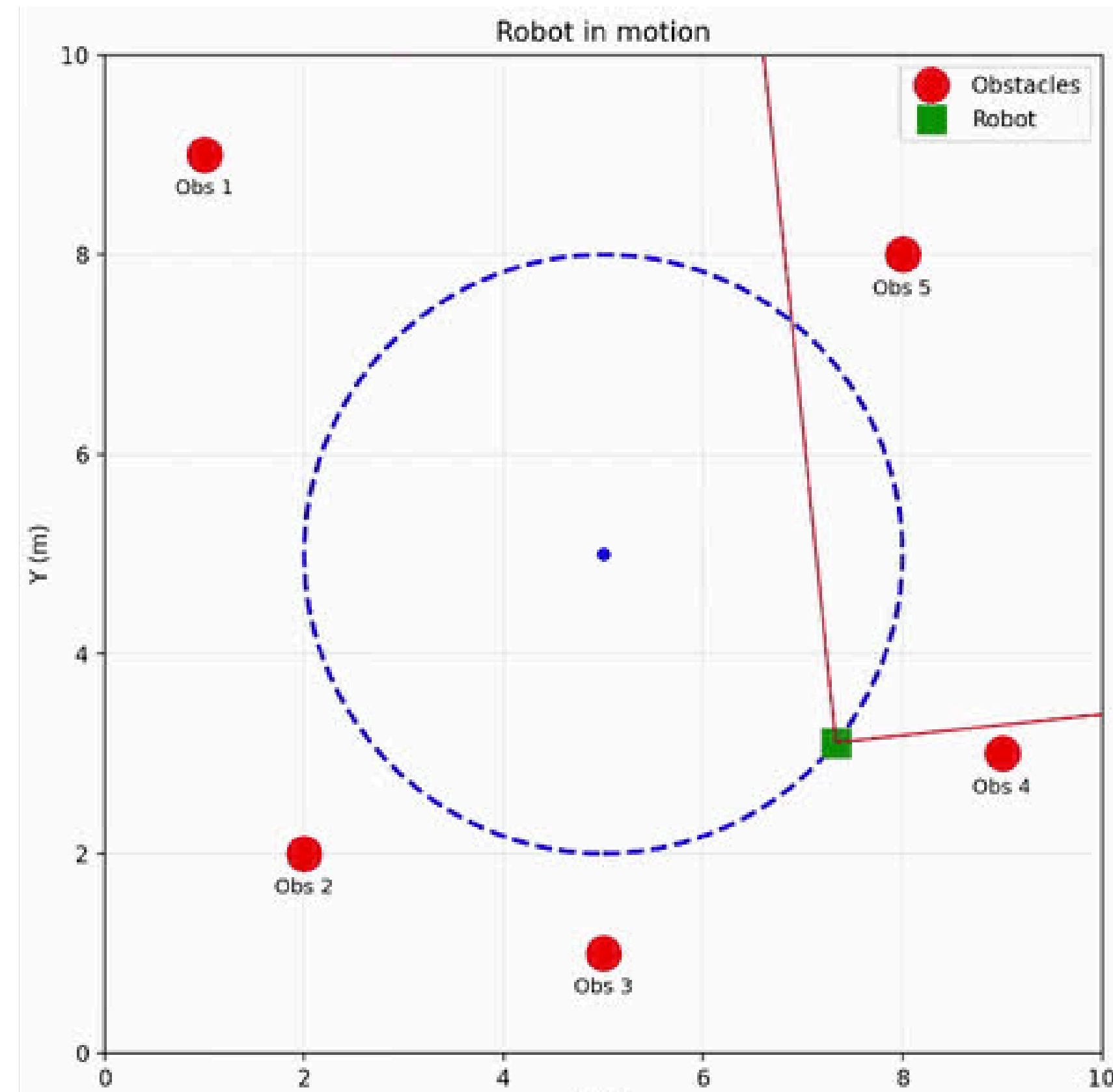
PERCEPTION

PATH PLANNING

ESTIMATION CHALLENGE

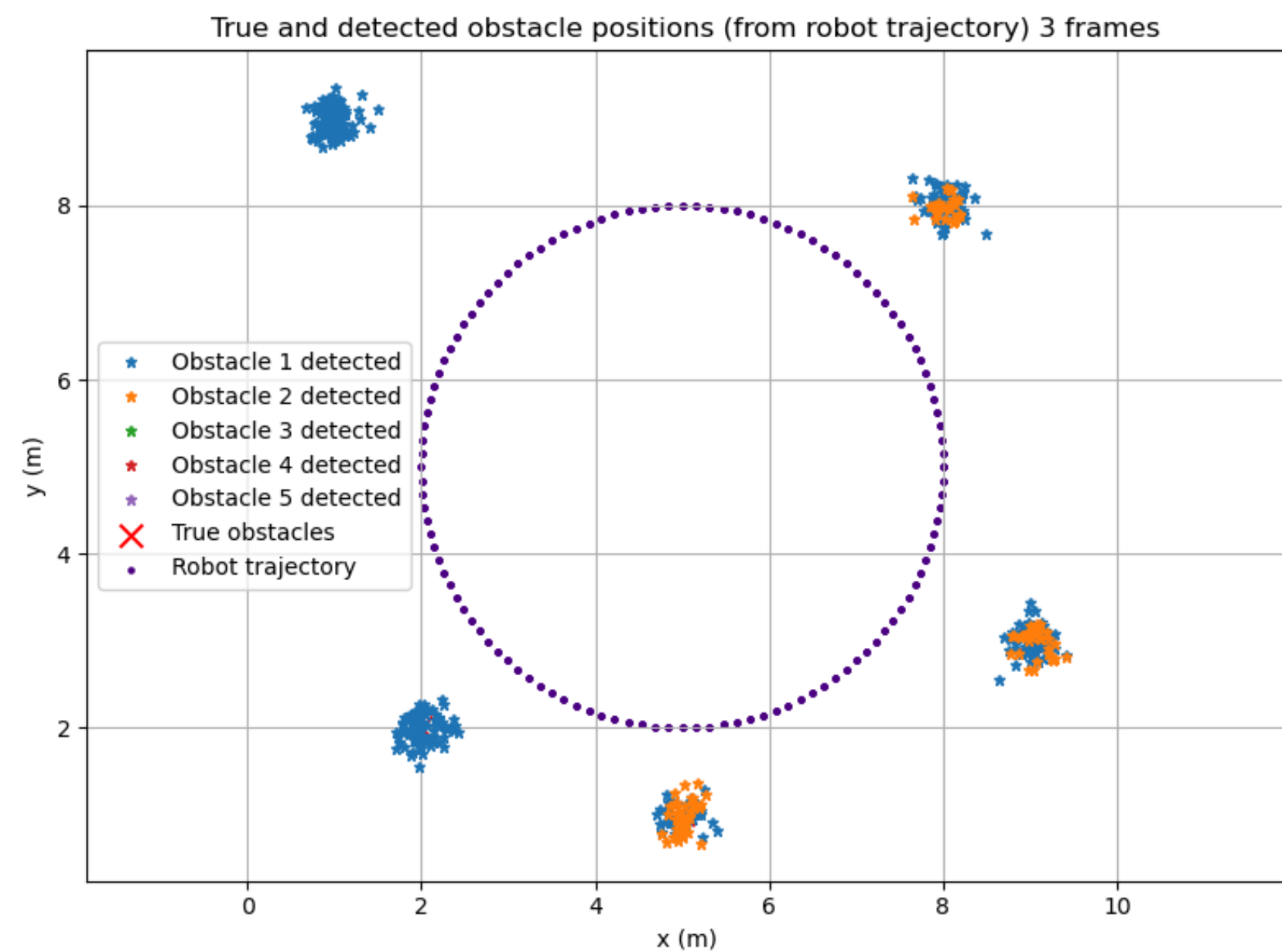
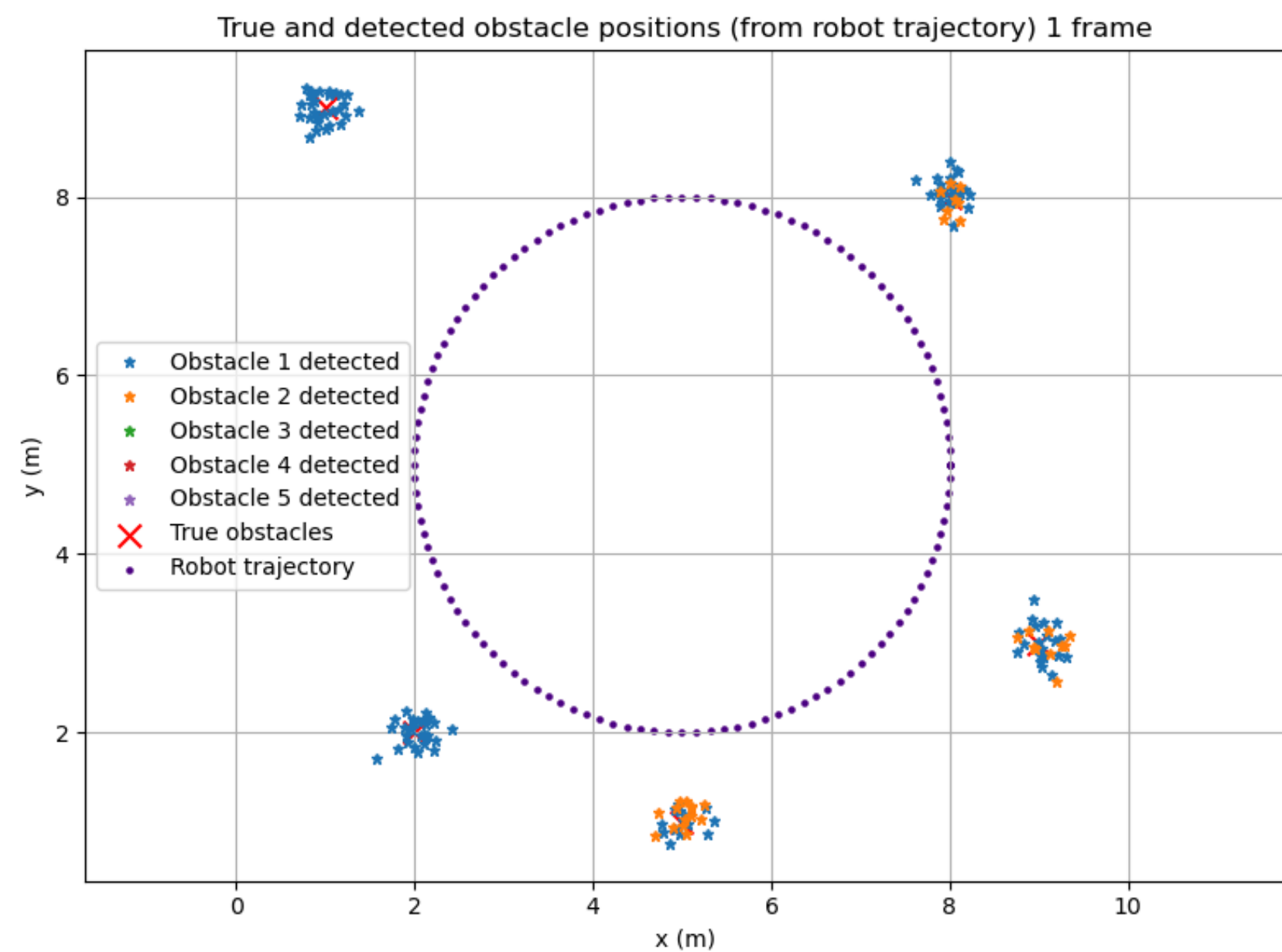
WHAT ARE WE GOING TO DO?

In this section, we will apply the Kalman filter to estimate the positions of obstacles based on noisy sensor measurements. This method allows us to combine the robot's observations with a static model to achieve more accurate and reliable position estimates.



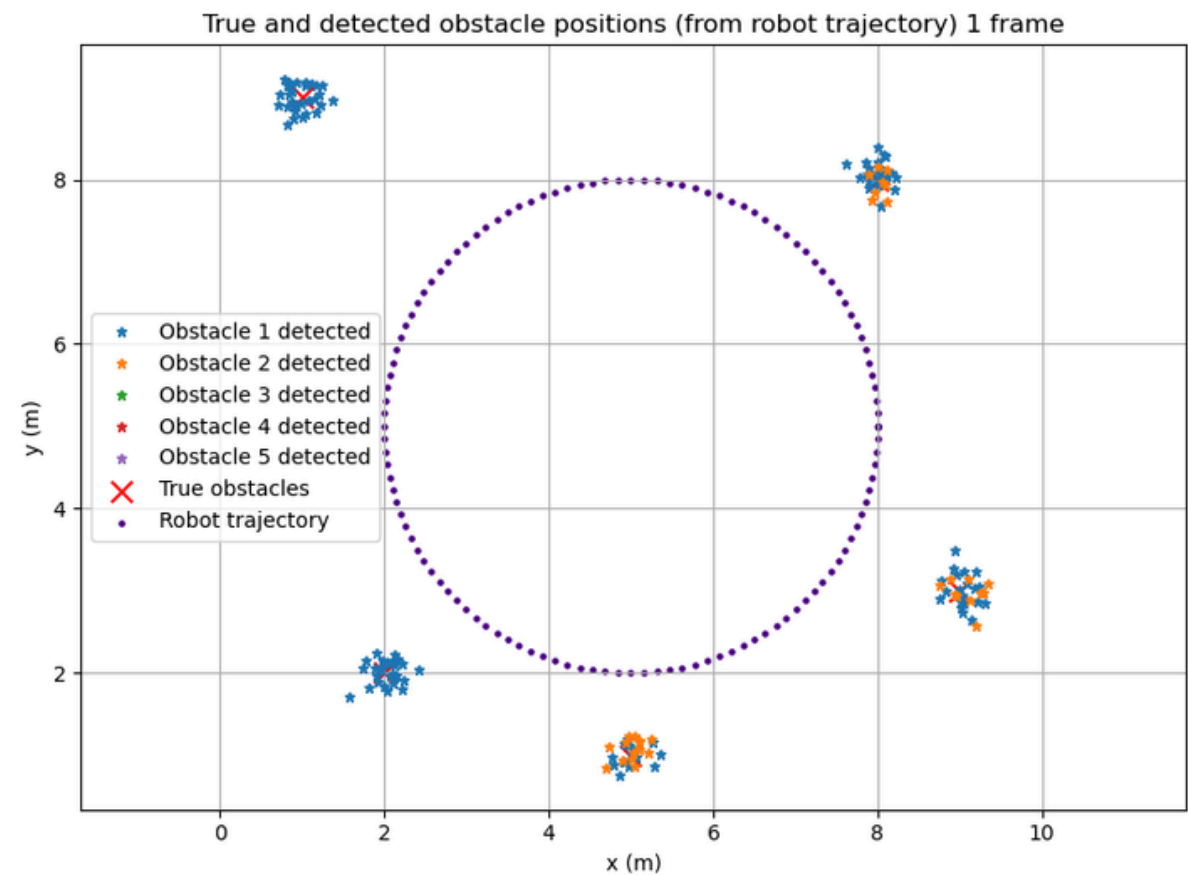
ESTIMATION

VISUALIZATION



ESTIMATION

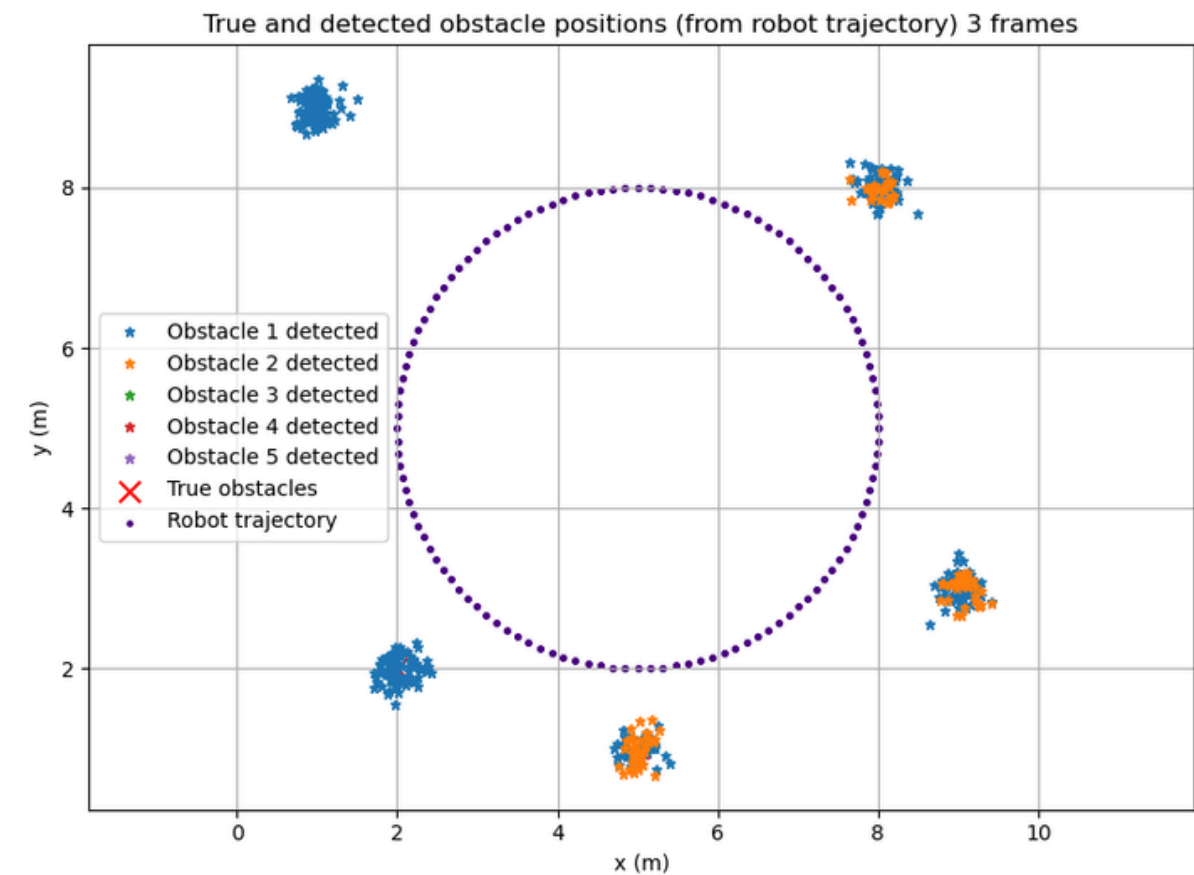
VISUALIZATION



ERRORS BY OBSTACLE (DISTANCE IN METERS):

- OBSTACLE 1 : 6.0948 M
- OBSTACLE 2 : 5.7290 M
- OBSTACLE 3 : NAN M
- OBSTACLE 4 : NAN M
- OBSTACLE 5 : NAN M

TOTAL ERROR (GLOBAL RMSE) : NAN M



ERRORS BY OBSTACLE (DISTANCE IN METERS):

- OBSTACLE 1 : 6.0704 M
- OBSTACLE 2 : 5.7182 M
- OBSTACLE 3 : NAN M
- OBSTACLE 4 : NAN M
- OBSTACLE 5 : NAN M

TOTAL ERROR (GLOBAL RMSE) : NAN M

ESTIMATION

DATA PROCESSING

Let the robot position at time step k be denoted by:

$$r_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}.$$

Let $D_k = \{(dx_1, dy_1), (dx_2, dy_2), \dots, (dx_M, dy_M)\}$ be the set of relative measurements from the robot to M obstacles at time k . Let $O_j = \begin{bmatrix} x_j \\ y_j \end{bmatrix}$ denote the true position of the j^{th} obstacle.

For each detection i , the absolute detected position of the obstacle at time k is given by:

$$z_k^{(i)} = r_k + \begin{bmatrix} dx_i \\ dy_i \end{bmatrix}.$$

For each detection i , we associate it with the nearest obstacle:

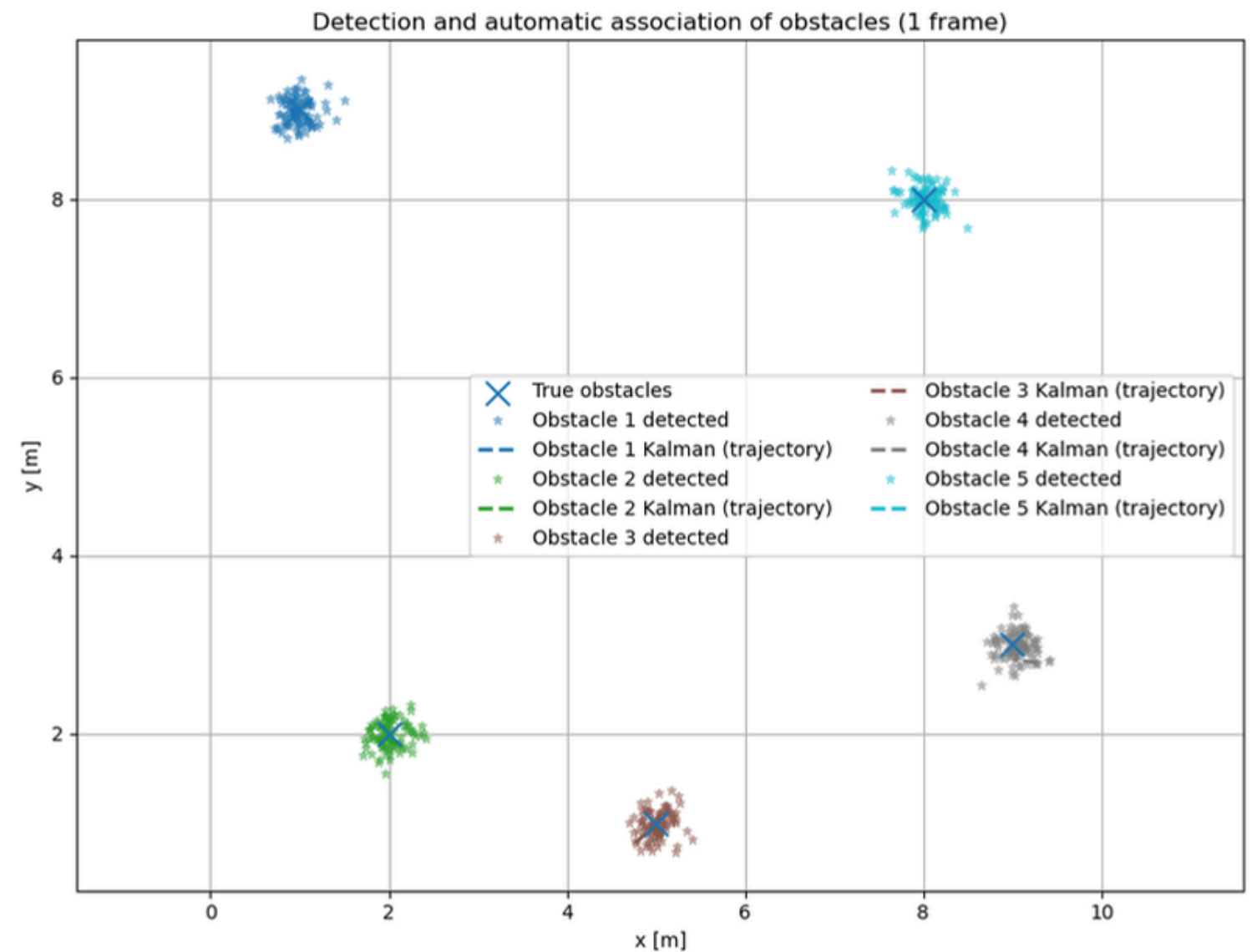
$$j_{\text{nearest}} = \arg \min_j \|z_k^{(i)} - O_j\|_2.$$

Thus, each detection is assigned to the closest obstacle in terms of spatial proximity.

Assign obstacle $O_{j_{\text{nearest}}}$ to detection $z_k^{(i)}$ if $d_{ij_{\text{nearest}}} = \min_j \|z_k^{(i)} - O_j\|_2$.

ESTIMATION

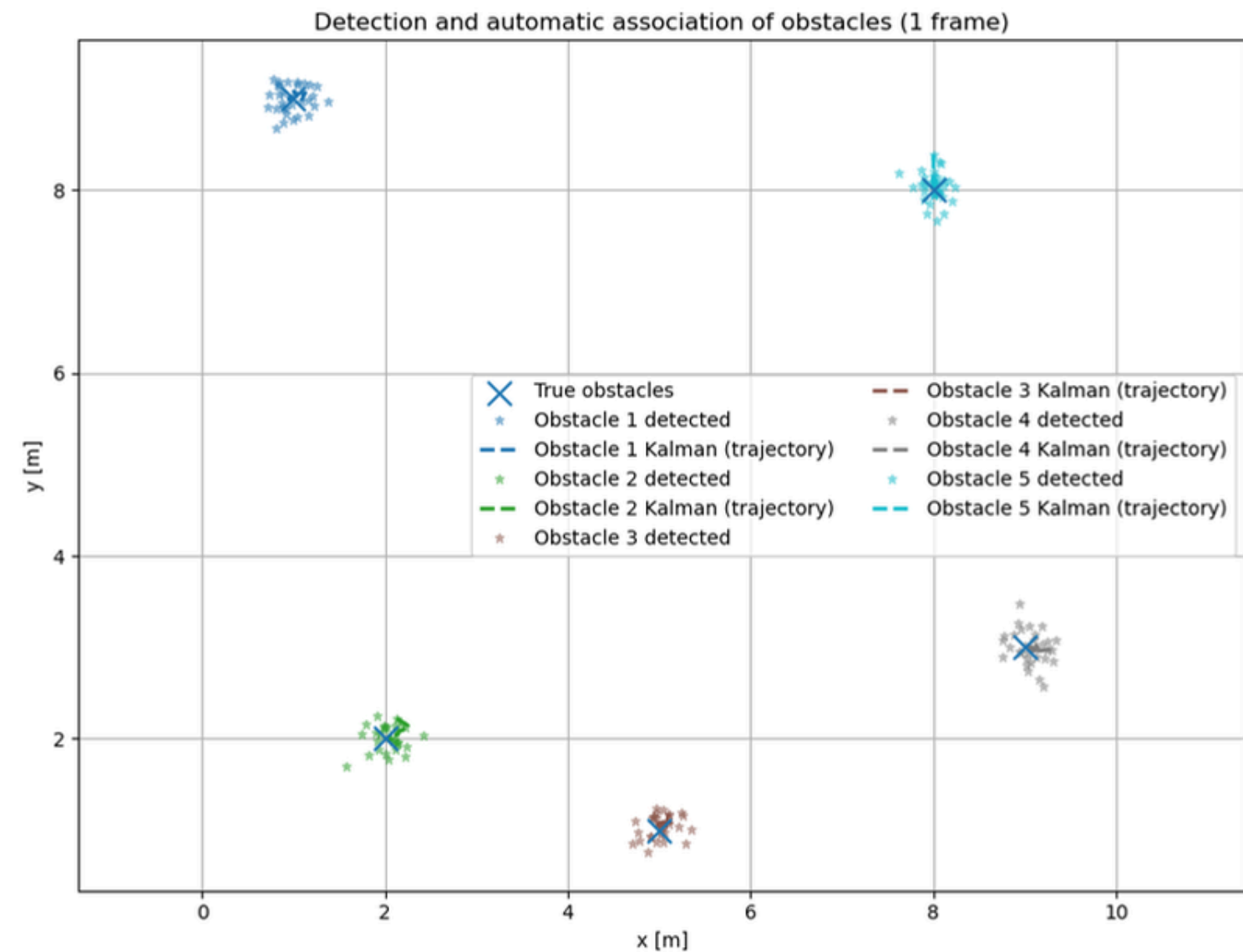
VISUALIZATION



DETECTION ERRORS (DISTANCE TO TRUE OBSTACLE):

- OBSTACLE 1: 0.0233 M
- OBSTACLE 2: 0.0076 M
- OBSTACLE 3: 0.0115 M
- OBSTACLE 4: 0.0402 M
- OBSTACLE 5: 0.0243 M

GLOBAL DETECTION RMSE: 0.0242 M



DETECTION ERRORS (DISTANCE TO TRUE OBSTACLE):

- OBSTACLE 1: 0.0204 M
- OBSTACLE 2: 0.0317 M
- OBSTACLE 3: 0.0392 M
- OBSTACLE 4: 0.0516 M
- OBSTACLE 5: 0.0338 M

GLOBAL DETECTION RMSE: 0.0368 M

ESTIMATION...

KALMAN FILTER

We aim to estimate the true position of a static obstacle:

$$x_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix} \in \mathbb{R}^2$$

from a sequence of noisy measurements:

$$z_k = \begin{bmatrix} z_k^x \\ z_k^y \end{bmatrix},$$

obtained at each time instant $k = 1, \dots, T$. The state evolution model is :

$$x_{k+1} = F x_k + w_k, \quad w_k \sim \mathcal{N}(0, Q),$$

and the observation model is:

$$z_k = H x_k + v_k, \quad v_k \sim \mathcal{N}(0, R).$$

ESTIMATION

KALMAN FILTER

In the case of a fixed obstacle :

$$F = I_2, \quad H = I_2, \quad Q = q_{\text{var}} I_2, \quad R = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix}.$$

Thus :

$$x_{k+1} = x_k + w_k,$$

$$z_k = x_k + v_k.$$

The Kalman filter provides, at each time k :

$$\hat{x}_{k|k} = \mathbb{E}[x_k \mid z_1, \dots, z_k], \quad P_{k|k} = \text{Cov}(x_k \mid z_1, \dots, z_k),$$

which is the optimal estimate in the sense of minimizing the quadratic variance in the linear-Gaussian case.

ESTIMATION...

KALMAN FILTER

1. Prédiction

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1|k-1},$$
$$P_{k|k-1} = FP_{k-1|k-1}F^\top + Q.$$

2. Innovation

$$y_k = z_k - H\hat{x}_{k|k-1},$$
$$S_k = HP_{k|k-1}H^\top + R.$$

3. Kalman gain

$$K_k = P_{k|k-1}H^\top S_k^{-1}$$

4. Update

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(z_k - H\hat{x}_{k|k-1}),$$
$$P_{k|k} = (I - K_kH)P_{k|k-1}.$$

ESTIMATION

KALMAN FILTER

The Kalman filter here acts as an adaptive weighted average:

$$\hat{x}_k = \hat{x}_{k-1} + K_k(z_k - \hat{x}_{k-1}),$$

where K_k weights the confidence between the prediction and the measurement:

$$K_k = \frac{P_{k-1}}{P_{k-1} + R}.$$

- If $R \ll P_{k-1}$: the measurements are reliable, the filter trusts them ($K_k \rightarrow I$).
- If $R \gg P_{k-1}$: the measurements are noisy, the filter favors the prediction ($K_k \rightarrow 0$).

ESTIMATION...

KALMAN FILTER

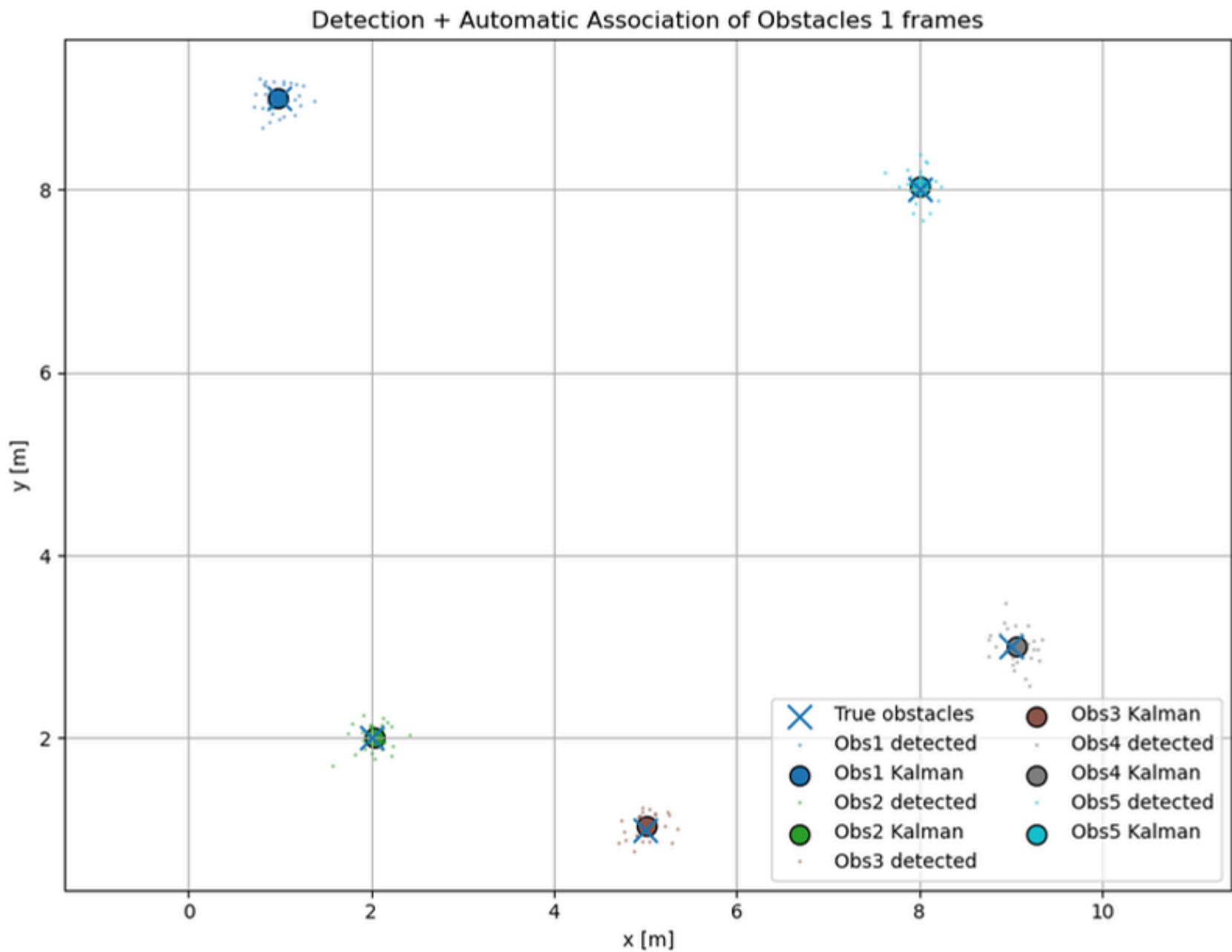
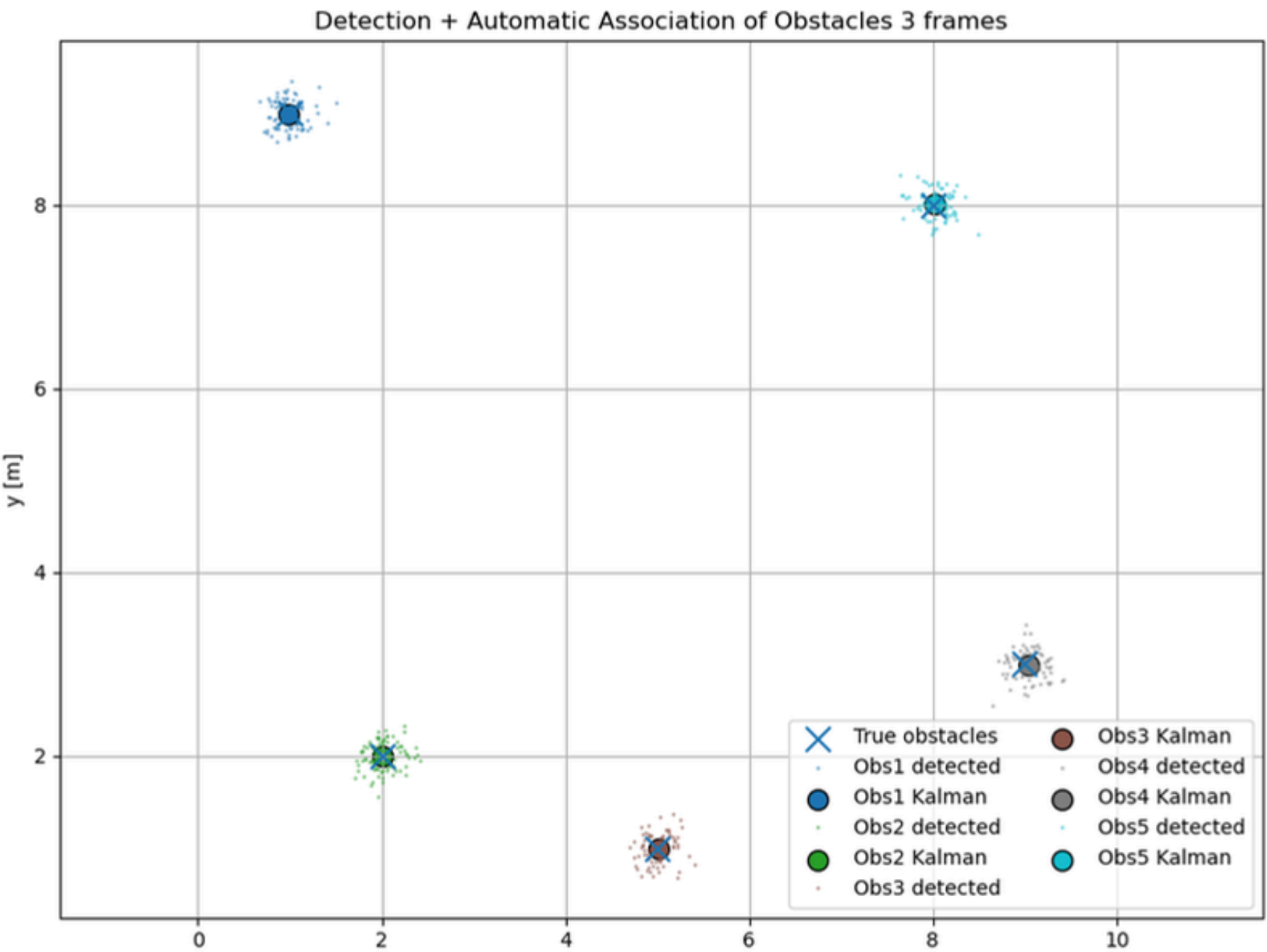
Final Result After convergence:

$$\hat{x}_{k|k} \approx \mathbb{E}[x \mid z_1, \dots, z_k], \quad P_{k|k} \rightarrow 0,$$

and the estimated position $\hat{x}_{k|k}$ becomes a very accurate estimate of the true position of the obstacle.

ESTIMATION...

AFTER KALMAN FILTER



ESTIMATION...

AFTER KALMAN FILTER

The measurement covariance matrix estimated from the data (one frame) is:

$$R_{\text{estimated}} = \begin{bmatrix} 0.00613236 & 0 \\ 0 & 0.00613236 \end{bmatrix}.$$

The estimation errors for each obstacle are:

$$\text{Errors per obstacle} = [0.0201, 0.0309, 0.0385, 0.0511, 0.0345].$$

The global Root Mean Squared Error (RMSE) is:

$$\text{RMSE}_{\text{global}} = 0.0365.$$

ESTIMATION...

AFTER KALMAN FILTER

The measurement covariance matrix estimated from the data (three frames) is:

$$R_{\text{estimated}} = \begin{bmatrix} 0.00561104 & 0 \\ 0 & 0.00561104 \end{bmatrix}.$$

The estimation errors for each obstacle are:

$$\text{Errors per obstacle} = [0.0231, 0.0076, 0.0145, 0.0341, 0.0298].$$

The global Root Mean Squared Error (RMSE) is:

$$\text{RMSE}_{\text{global}} = 0.0239.$$

ESTIMATION...

COMMENTS

Number of Detection	Detection RMSE [m]	Kalman Filter RMSE [m]
123	0.0368	0.0365
363	0.0242	0.0239

- The Kalman filter produces slightly lower RMSE
- It can also be observed that the RMSE decreases as the number of detections increases

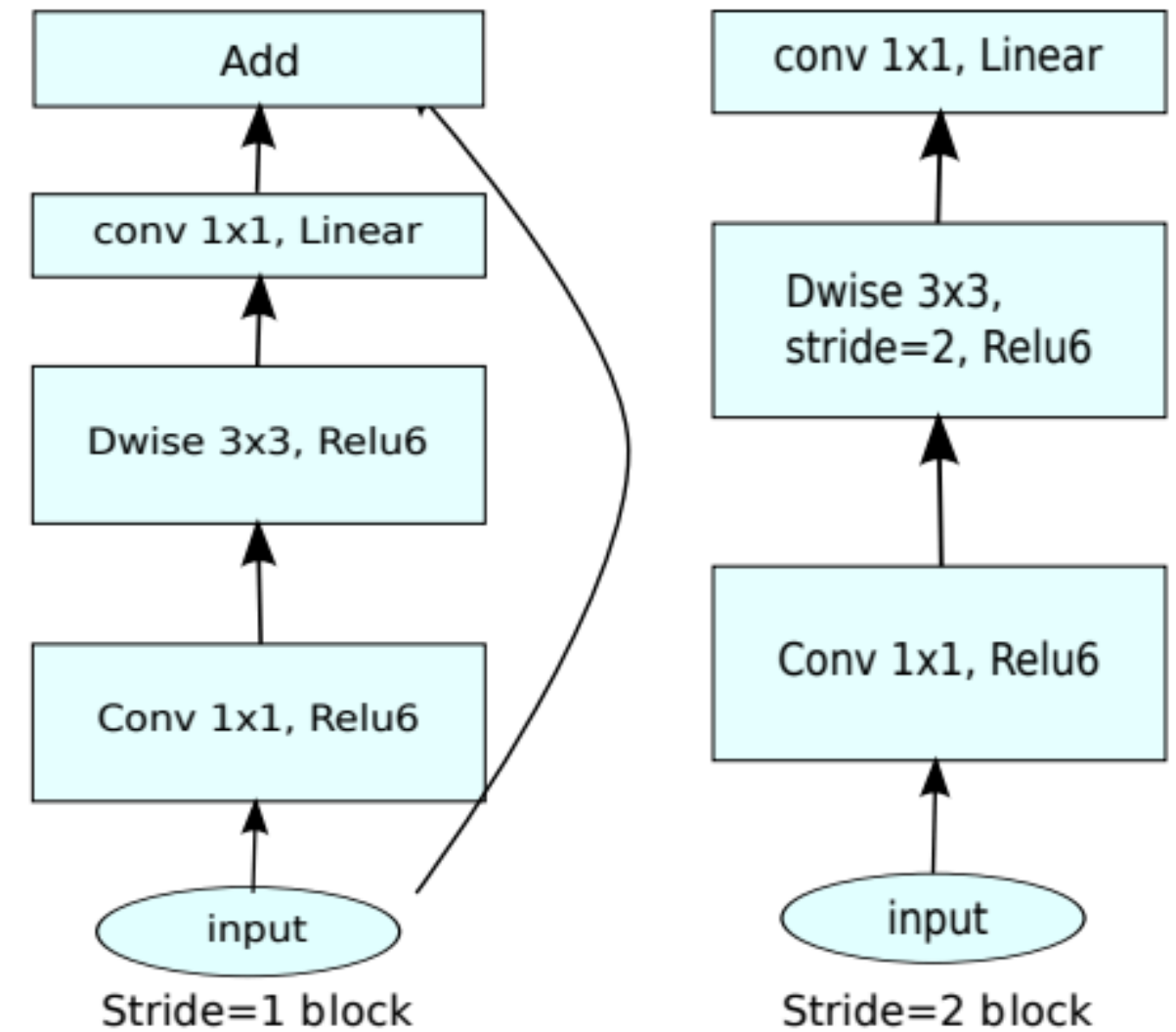
PERCEPTION CHALLENGE

THE CHALLENGE

The challenge is to fine-tune the pre-trained MobileNetV2 model to classify images from the CIFAR-100 dataset.

Objectives:

- >90% accuracy
- The simpler the better



PERCEPTION

CONCEPTS

MobileNetV2:

A lightweight convolutional neural network developed by Google, originally trained on ImageNet ($\approx 14\text{M}$ images) for image classification tasks.

Fine-tuning:

The process of taking a pre-trained model and adapting it to a new dataset by retraining some or all of its layers.

PERCEPTION

METHODOLOGY

- 1 Preprocess and augment the dataset.**
- 2 Freeze most layers of the pre-trained MobileNetV2.**
- 3 Add trainable layers to improve performance.**

PERCEPTION

TOOLS



TensorFlow and Keras: loading dataset, creating and training the model.

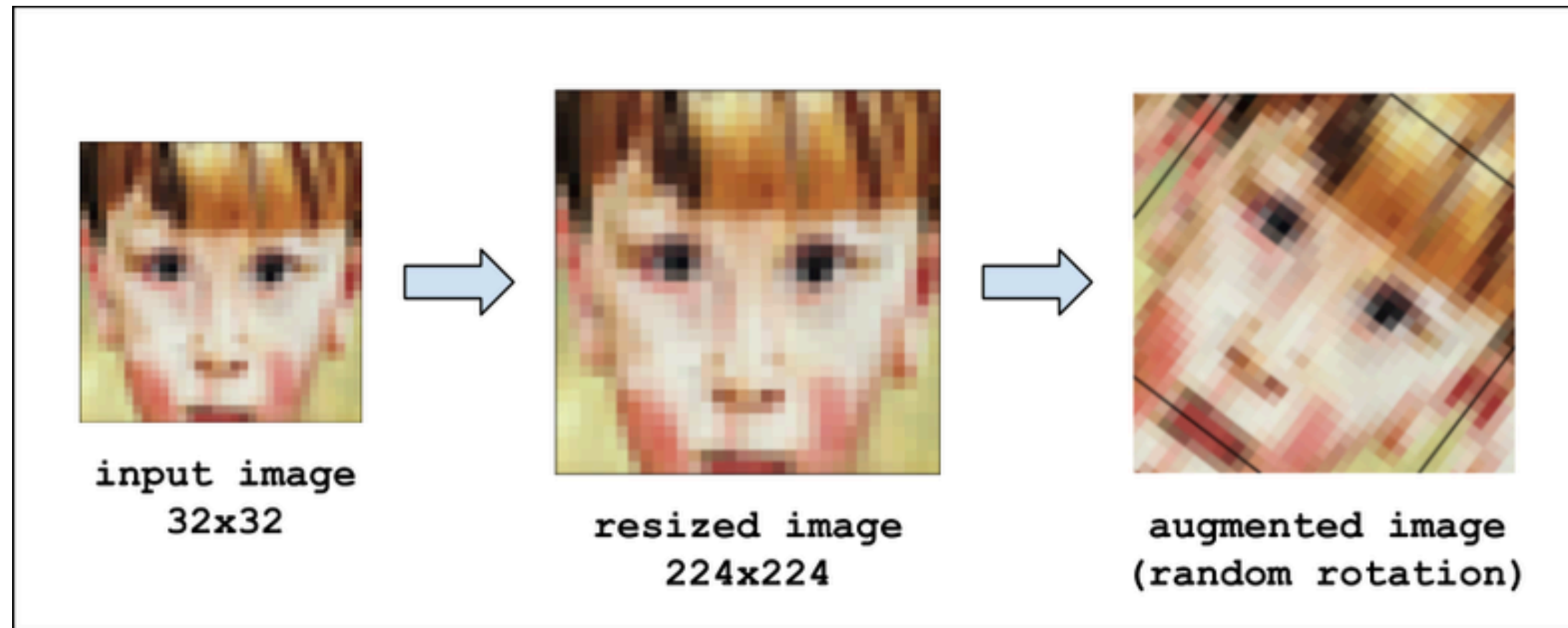


Jupyter Notebook: writing interactive, documented code.

PERCEPTION

PRE-PROCESSING & DATA AUGMENTATION

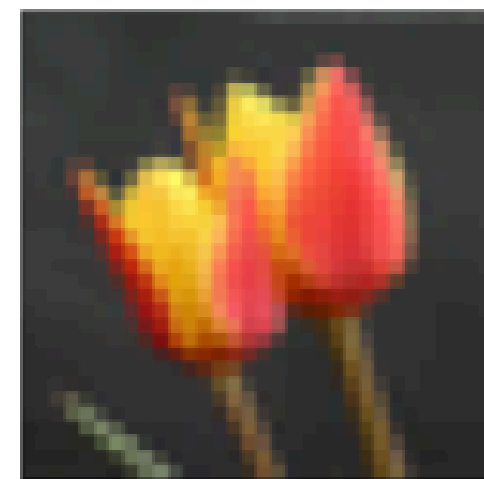
The CIFAR-100 dataset is composed of 32x32 images, but the MobileNetV2 model expects 224x224 inputs → **resize the inputs**



PERCEPTION

PRE-PROCESSING & DATA AUGMENTATION

Data augmentation:
artificially increasing the
size and diversity of a
training dataset by
applying random
transformations to
existing images
(rotations, flips, zooms,
translations, color
adjustments...)



original image



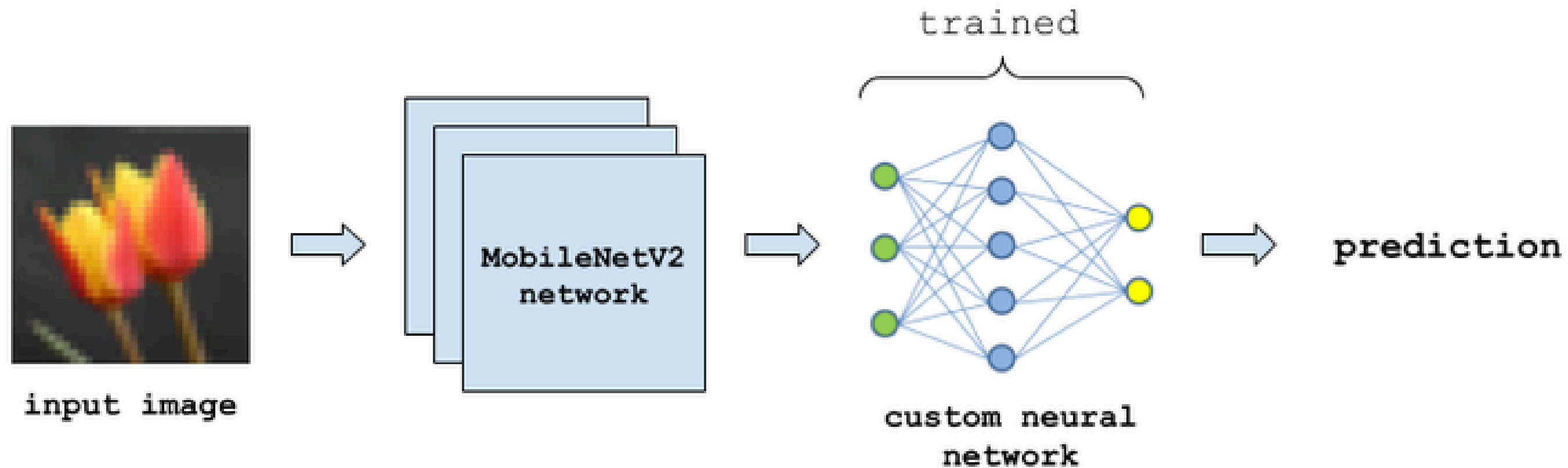
possible variations

PERCEPTION

MODEL ARCHITECTURE

1st Approach:

Keeping all the weights for the base model and adding a few layers on top.

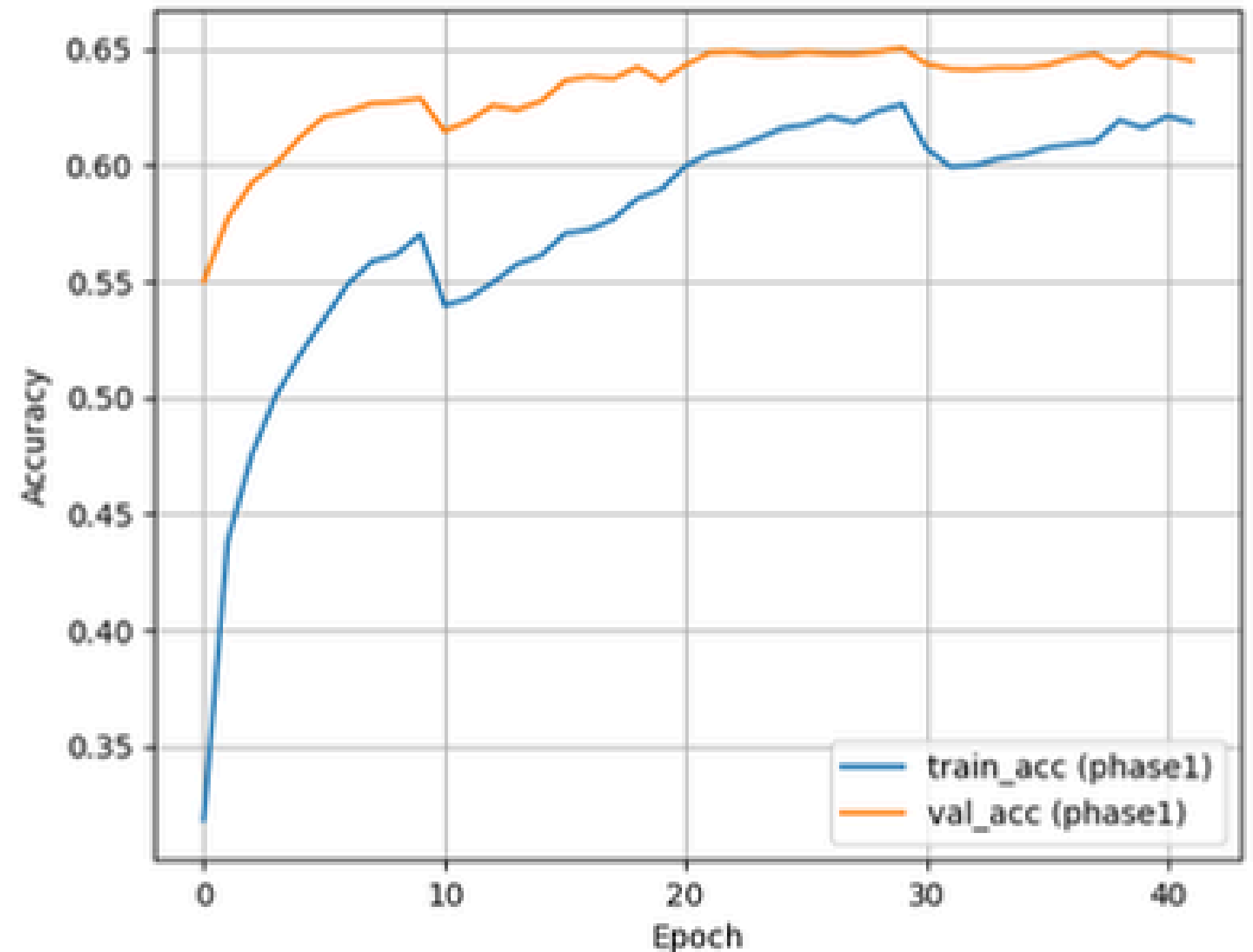


PERCEPTION

MODEL ARCHITECTURE

Results: 1st approach

Accuracy: ~65% on validation set (40 epochs) → **not good enough, we need to go deeper**

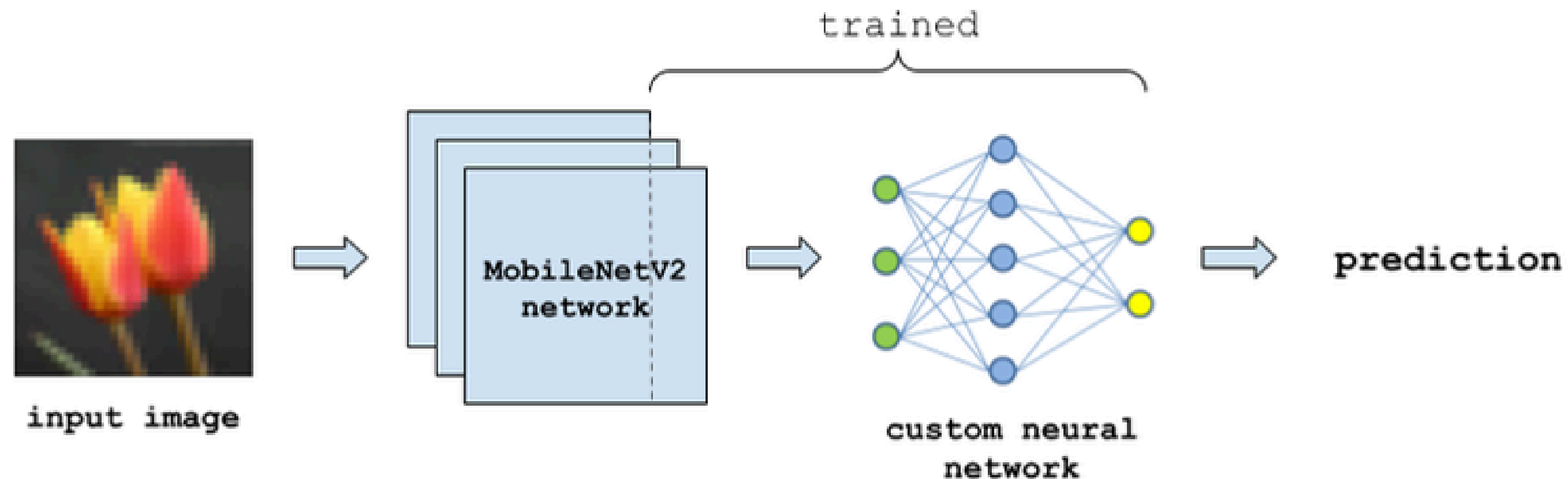


PERCEPTION

MODEL ARCHITECTURE

2nd Approach:

Freezing most MobileNetV2 layers except the last 30, and adding a few layers on top



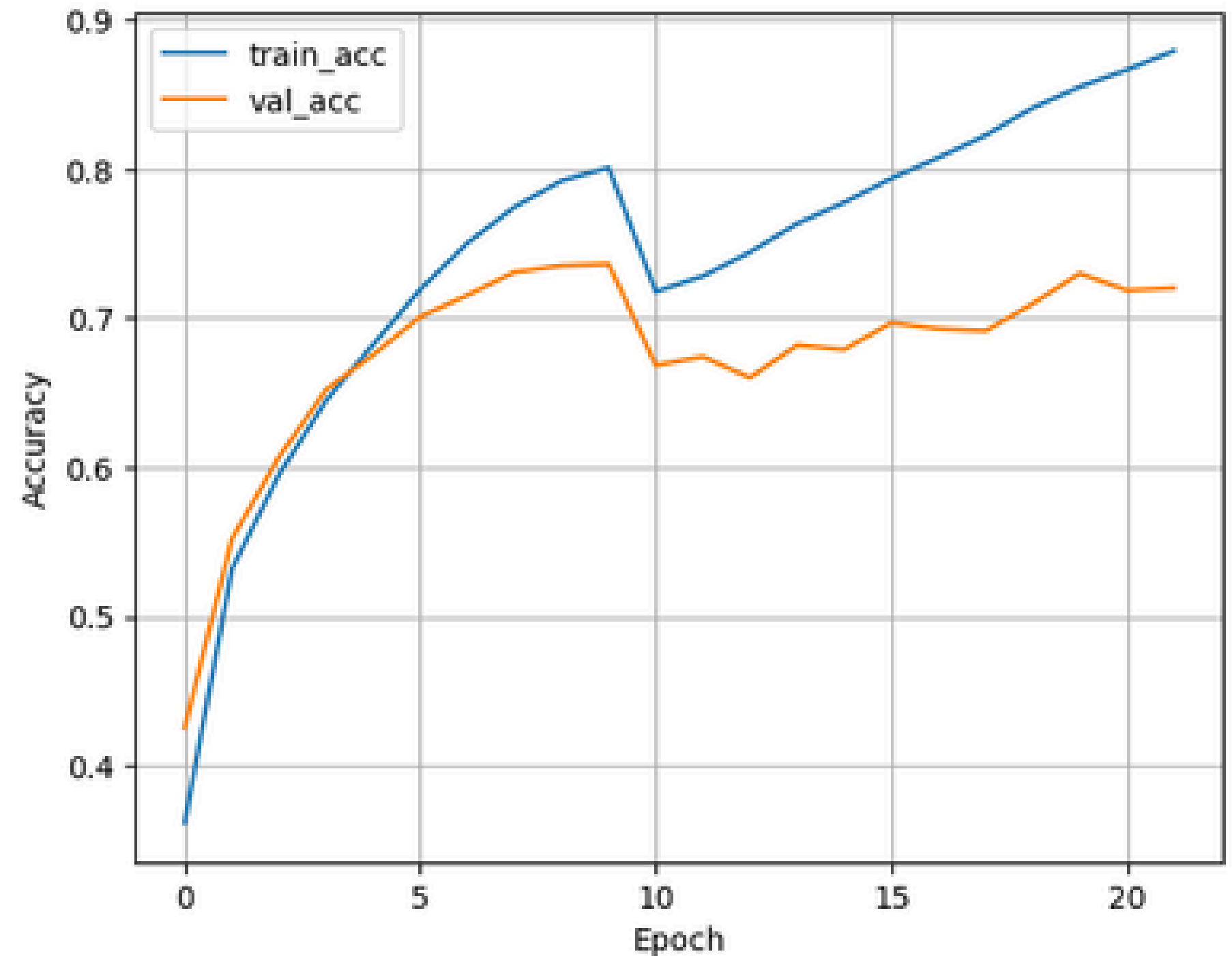
PERCEPTION

MODEL ARCHITECTURE

Results: 2nd approach

Accuracy: ~74% on validation set (~20 epochs)

→ **significantly better result**



PERCEPTION

MODEL ARCHITECTURE

Model summary :

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 224, 224, 3)	0
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
gap (GlobalAveragePooling2D)	(None, 1280)	0
batch_normalization (BatchNormalization)	(None, 1280)	5,120
dense (Dense)	(None, 1024)	1,311,744
batch_normalization_1 (BatchNormalization)	(None, 1024)	4,096
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 512)	524,800
batch_normalization_2 (BatchNormalization)	(None, 512)	2,048
dropout_1 (Dropout)	(None, 512)	0
dense_2 (Dense)	(None, 100)	51,300

PERCEPTION

TRAINING

Hyperparameters:

Number of epochs: 60 (with early-stopping)

Batch size: 64

Initial learning rate: 0.0005

Trainable parameters:

3,419,876

Hardware:

NVIDIA GeForce RTX 3050

Mobile GPU

Training took ~90 minutes for 40 epochs

PERCEPTION

COMMENTS

Training the last 30 vs 40 layers of MobileNetV2 yielded very little improvement → **overfitting?**

The best accuracy achieved was 74%

PERCEPTION

PATH-PLANNING CHALLENGE

THE CHALLENGE



In this challenge, we will develop a fast and reliable path-planning module for a driverless car that computes smooth driving paths from cone detections.

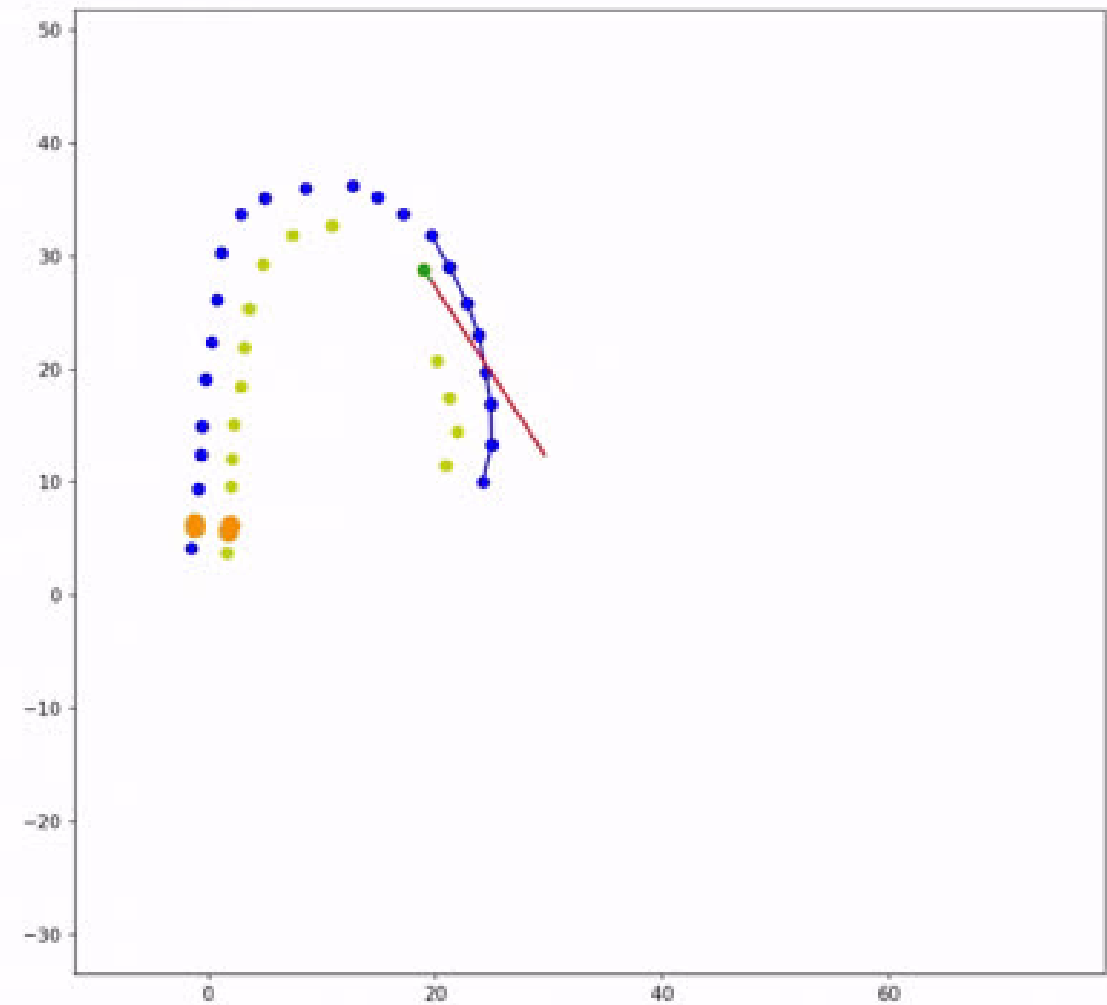
Objectives :

- Improve Computing time
- Complexity and performance of the algorithm used
- Viability to be driven (smooth line, stable cornering)

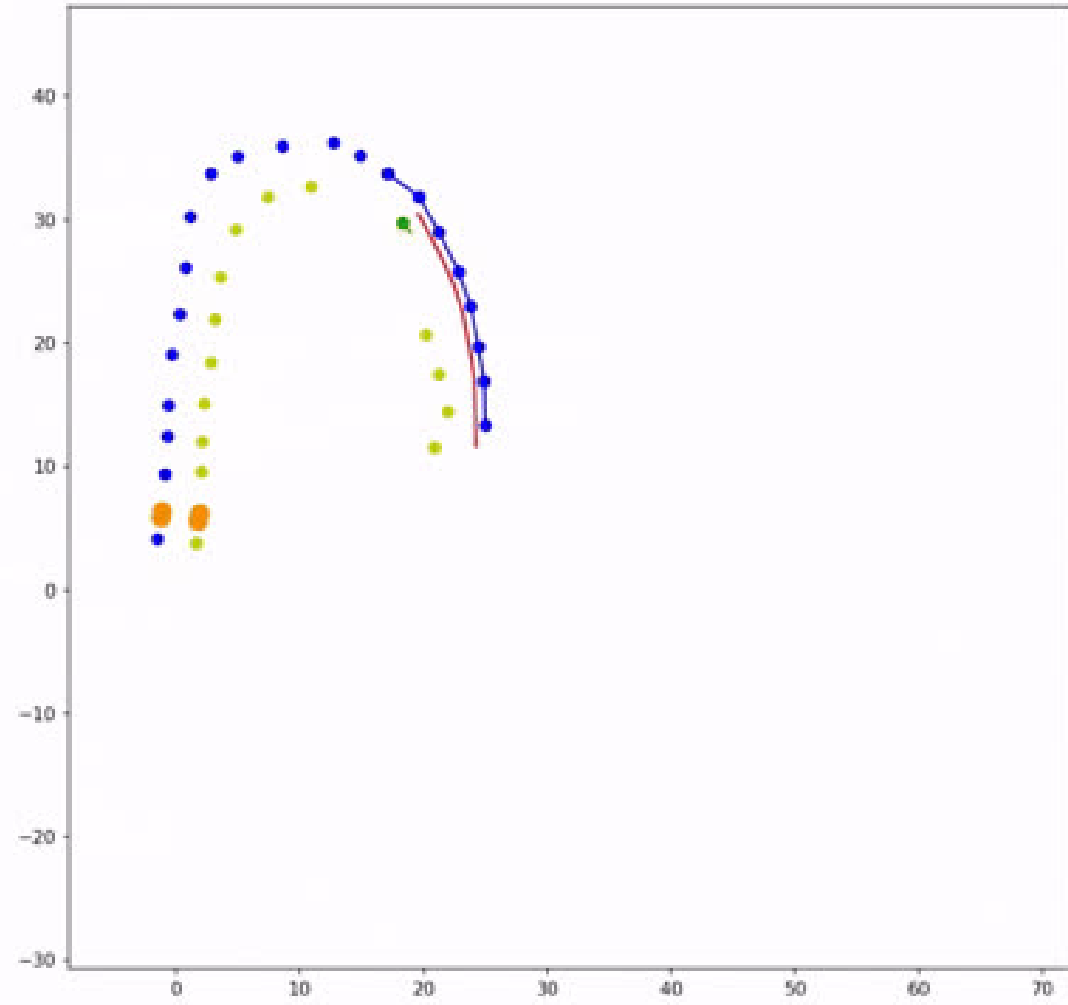
PATH-PLANNING

VISUALIZATION

—



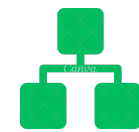
FRAME 1 TOOK 0.1245 SECONDS



FRAME 1 TOOK 0.1359 SECONDS

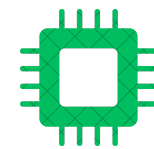
PATH-PLANNING

ALGORITHM OVERVIEW



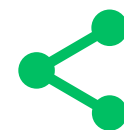
SELECT_SIDE_TO_USE

Selects the reference side (left/right) based on the **number and quality** of cone matches; tie-break with the **sum of indices**.



CALCULATE_CENTERLINE_POINTS

Computes **midpoints** between cones on the reference side and their matches on the opposite side.



FIT_MATCHES_AS_SPLINE

Fits a **C² spline** through the raw midpoints to create a baseline path; **falls back** to the previous path on failure.

PATH-PLANNING

ALGORITHM OVERVIEW

CONNECT_TO_EGO_IF_AHEAD_AND_FAR

Inserts a short **link point** only if the first path point is **ahead** of the ego and **farther than ~0.5 m**.

REMOVE_PATH_BEHIND_CAR

Finds the closest point on the path and **drops all points behind** the car to keep a **forward look-ahead** only.

EXTEND_PATH

Extends the path to reach *mpc_path_length*: try a **circular-arc** (circle fit on the tail), otherwise **linear** extension.

PATH-PLANNING

I/O VARIABLES

INPUTS:

- The car's current position and orientation in the map : `position, direction (2,)`
- The position of the colored (right/left) cones in the map : `left_cones, right_cones (N, 2)`

OUTPUTS:

- `path_parameterized (N, 4)` → samples of a parameterized B-spline ready for MPC
with columns: `theta, x, y, curvature`

PATH-PLANNING

EQUATIONS

— SELECT_SIDE_TO_USE

CALCULATE_CENTERLINE_POINTS

$$S \in \{L, R\}, \quad I_S = \{i \mid m_i^S \geq 0\}, \quad A_S = |I_S|, \quad B_S$$

$$S^* = \arg \max_S (A_S, B_S) \quad (\text{maximize } A, \text{ tie-break } B)$$

$$\forall i \in I_{S^*} : \quad c_i = \frac{1}{2} \left(p_i^{S^*} + p_{m_i^{S^*}}^{\bar{S}^*} \right)$$

$$\bar{L} = R, \quad \bar{R} = L$$

PATH-PLANNING

EQUATIONS

— **FIT_MATCHES_AS_SPLINE**

OBJECTIVE:

$$P = \text{predict}_{\text{der}=0}(\text{spline_fit}(\{c_i\}_{i=1}^M))$$

SETTINGS:

`smoothing =  $\lambda$` , `max_degree` and `predict_every` (sampling density)

FIT FAILURE:

reuse the last valid path

PATH-PLANNING

MPC

—

LOOP:

- Predicts the car's motion over **N** steps and chooses controls that **follow the path** while respecting limits. apply only the first control and repeat.

INPUT:

- Current state + path samples (θ, x, y, κ) over the prediction horizon.

PATH-PLANNING

COMPLEXITY

VECTOR :

- Use NumPy primitives (no Python loops): np.cumsum, np.linalg.norm(axis=1), np.argmin, np.clip, np.arange/linspace.
- Use SciPy : splprep/splev $\approx$ linear in M (fit) and K_{eval} (eval) at our sizes.

OVERALL :

$$\text{Time} \approx \mathcal{O}(M + K_{\text{eval}} + N)$$

N_L, N_R : cones; M : midpoints; $K_{\text{eval}} \approx \text{max_u/predict_every}$; $N = \text{mpc_prediction_horizon}$.

**THANK YOU FOR YOUR
ATTENTION!**

